

Department of Computer and Software Engineering – ITU

CE101T: Object Oriented Programming

Course Instructor: Nadir Abbas	Dated: 02/03/2025
Teaching Assistant: Zainab, Nahal, Bilal	Semester: Spring 2025
Session: 2024-2028	Batch: BSCE2024

Assignment 6. Implementation Of Inheritance And Its Levels

Name	Roll number	Obtained Marks/35

Checked on: _____

Signature: _____

Submission:

- Email instructor or TA if there are any questions. You cannot look at others' solutions or use others' solutions, however, you can discuss it with each other. Plagiarism will be dealt with according to the course policy.
- Submission after due time will not be accepted.

In this assignment you have to do following tasks:

Task 1: Ensure that you have installed all three softwares in your personal computer (Github, Cygwin & CLion). Now, accept the assignment posted in classroom (e.g Google, LMS etc) and after accepting, clone the repository to your computer. Make sure you have logged into the github app with your account.

Task 2: Open Cygwin app, Move to your code directory with following command “cd <path_of_folder>”

<path_of_folder> can be automatically populated by dragging the folder and dropping it to the cygwin window.

Run the code through Cygwin, use command “make run”, to get the output of the code

Task 3: Solve the given problems, write code using **CLion** or any other IDE.

Task 4: Keep your code in the respective git cloned folder.

Task 5: Commit and Push the changes through the Github App

Task 5: Write the code in separate files (**as instructed**). Ensure that file names are in lowercase (e,g **main.cpp**).

Task 6: Run ‘**make run**’ to run C++ code

Task 7: Run ‘**make test**’ to test the C++ code

Q1: Implement Class Hierarchy

Write the implementation for the following classes:

1. Person

- **Members:**
 - string name
 - int age
- **Constructors & Methods:**
 - Person(string name, int age);
 - string getName() const;
 - void setName(string name);
 - int getAge() const;
 - void setAge(int age);

2. Student (inherits Person)

- **Members:**
 - string studentID
 - double GPA
- **Constructors & Methods:**
 - Student(string name, int age, string studentID, double GPA);
 - string getStudentID() const;
 - void setStudentID(string studentID);
 - double getGPA() const;
 - void setGPA(double GPA);

3. Teacher (inherits Person)

- **Members:**
 - string teacherID
 - double salary
- **Constructors & Methods:**
 - Teacher(string name, int age, string teacherID, double salary);
 - string getTeacherID() const;
 - void setTeacherID(string teacherID);
 - double getSalary() const;
 - void setSalary(double salary);

4. UndergraduateStudent (inherits Student)

- **Members:**
 - int yearOfStudy
- **Constructors & Methods:**
 - UndergraduateStudent(string name, int age, string studentID, double GPA, int yearOfStudy);
 - int getYearOfStudy() const;
 - void setYearOfStudy(int yearOfStudy);

5. PostgraduateStudent (inherits Student)

- **Members:**

- string researchTopic
- **Constructors & Methods:**
 - PostgraduateStudent(string name, int age, string studentID, double GPA, string researchTopic);
 - string getResearchTopic() const;
 - void setResearchTopic(string researchTopic);
- 6. **TeachingAssistant** (inherits both Student and Teacher)
 - **Members:**
 - int hoursWorked
 - **Constructors & Methods:**
 - TeachingAssistant(string name, int age, string studentID, double GPA, string teacherID, double salary, int hoursWorked);
 - int getHoursWorked() const;
 - void setHoursWorked(int hoursWorked);
- 7. **AssistantProfessor** (inherits Teacher)
 - **Members:**
 - int publicationCount
 - **Constructors & Methods:**
 - AssistantProfessor(string name, int age, string teacherID, double salary, int publicationCount);
 - int getPublicationCount() const;
 - void setPublicationCount(int publicationCount);

Q2: Implement a Function to Calculate GPA Category

Class: Student

Implement a function **string getGPAcategory()** that evaluates the student's **GPA** and returns one of the following categories:

- **"Excellent"** if GPA is ≥ 3.7
- **"Good"** if GPA is **between 3.0 and 3.69**
- **"Average"** if GPA is **between 2.0 and 2.99**
- **"Poor"** if GPA is **below 2.0**

Q3: Implement a Function to Increase Salary

Class: Teacher

- Implement void **increaseSalary(double percentage)** that increases the salary by a given percentage.

Q4: Implement a Function to Check Research Eligibility

Class: PostgraduateStudent

- Implement **bool isEligibleForResearch()** that returns true if GPA is **greater than 3.5**, otherwise false.

Q5: Implement a Function to Count Total Working Hours

Class: TeachingAssistant

- Implement double **getTotalHours(int weeks)** that returns **total hours worked** given the number of weeks.

Q6: Implement a Function to Get Student Year Group

Class: UndergraduateStudent

Implement **string getYearGroup()** that returns:

- **"Freshman"** if yearOfStudy == 1
- **"Sophomore"** if yearOfStudy == 2
- **"Junior"** if yearOfStudy == 3
- **"Senior"** if yearOfStudy == 4 or more

Q7: Implement a Function to Calculate Bonus for Assistant Professors

Class: AssistantProfessor

- Implement **double calculateBonus()** that returns **10% of salary** if publication count > 10, otherwise **5% of salary**.

Q8: Implement a Function to Determine Student Type

Class: Student

- Implement **bool determineStudentType()** that determines whether the student is Undergraduate or Postgraduate and **returns 0 if Undergraduate and 1 if Postgraduate**.

Q9: UML Diagram

- Draw a UML class diagram for the above **inheritance hierarchy**.
- Show all class relationships (inheritance, members, methods).
- Save it as UML_diagram.png and include it in the submission folder.

Assessment Rubric for Assignment

Performance metric	CL O	Able to complete the task over 80% (4-5)	Able to complete the task 50-80% (2-3)	Able to complete the task below 50% (0-1)	Marks
1. Realization of experiment	3	Executes without errors excellent user prompts, good use of symbols, spacing in output. Through testing has been completed.	Executes without errors, user prompts are understandable, minimum use of symbols or spacing in output. Some testing has been completed.	Does not execute due to syntax errors, runtime errors, user prompts are misleading or non-existent. No testing has been completed.	
2. Conducting experiment	2	Able to make changes and answered all questions.	Partially able to make changes and few incorrect answers.	Unable to make changes and answer all questions.	
3. Computer use	4	Document submission timely.	Document submission late.	Document submission not done.	
4. Teamwork	4	Actively engages and cooperates with other group member(s) in effective manner.	Cooperates with other group member(s) in a reasonable manner but conduct can be improved.	Distracts or discourages other group members from conducting the experiment	
5. Laboratory safety and disciplinary rules	2	Code comments are added and does help the reader to understand the code.	Code comments are added and does not help the reader to understand the code.	Code comments are not added.	
6. Data collection	2	Excellent use of white space, creatively organized work, excellent use of variables and constants, correct identifiers for constants, No line-wrap.	Includes name, and assignment, white space makes the program fairly easy to read. Title, organized work, good use of variables.	Poor use of white space (indentation, blank lines) making code hard to read, disorganized and messy.	
7. Data analysis	3	Solution is efficient, easy to understand, and maintain.	A logical solution that is easy to follow but it is not the most efficient.	A difficult and inefficient solution.	
Total (out of 35):					